Timing Specifications

- CMSC313
- Dr. Kidd

Acknowledgements

- Parts of these lectures are taken or derived from one or more of the following:
- Harris and Harris, Digital Design and Computer Architecture, RISC5 Edition
- Victor P. Nelson, Carroll, Nagle, Irwin, Digital Logic Circuit Analysis and Design.
- M. Morris Mano, Digital Logic and Computer Design.

Timing Specifications

t_{su} – setup time
- Min interval the inputs must be stable before the trigger

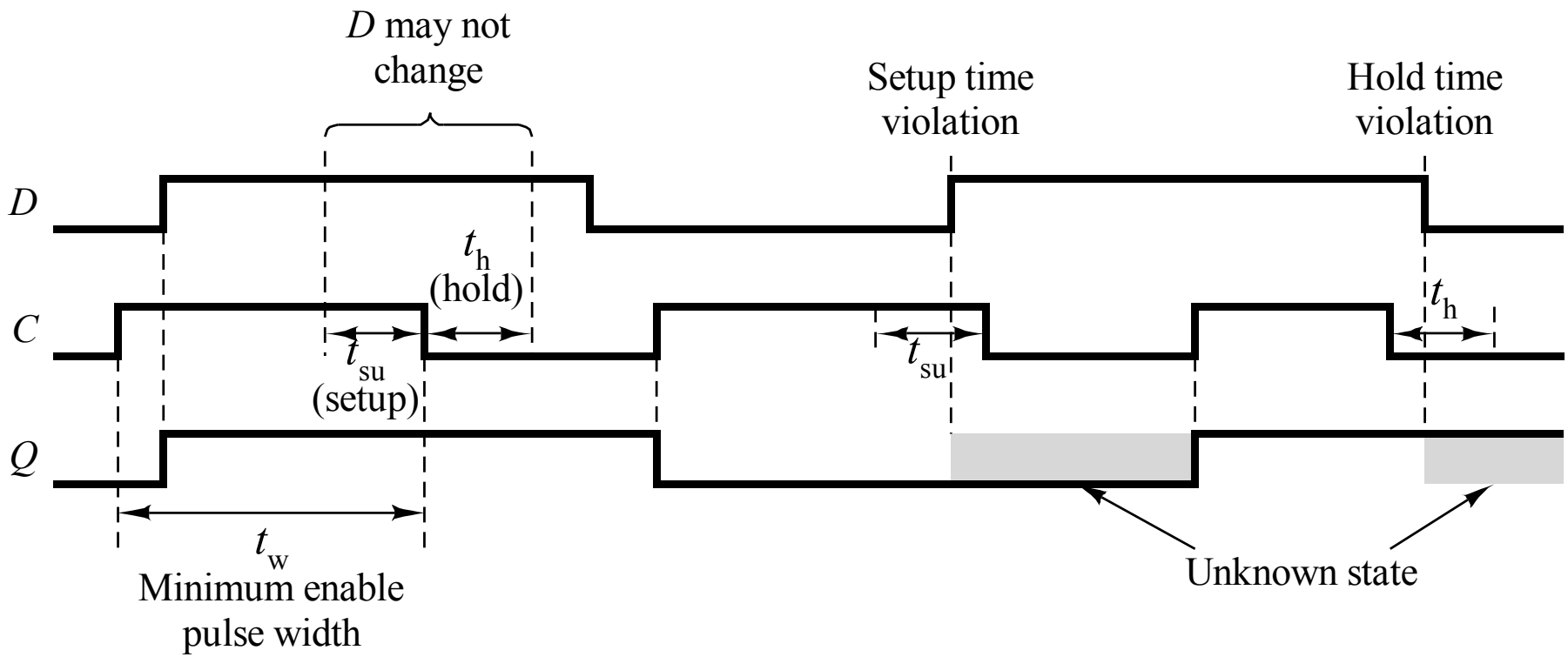
t_h – hold time
- Min interval the inputs must be held after the trigger

t_w – pulse width
- Min interval an input must be held

t_{PLH} – propagation low to high
- Max interval for a low input to result in a high output

t_{PHL} – propagation high to low
- Maximum interval for a high input to result in a low output

D-Latch Timing Constraints



Propagation Delays and Time Constraints for the SN74LS75

TABLE 6.2 PROPAGATION DELAYS AND TIME CONSTRAINTS FOR THE SN74LS75 [1]

Timing Parameter	Typical Value (ns)
t_{PLH} $(D \rightarrow Q)$	15
t_{PHL} $(D \rightarrow Q)$	9
t_{PLH} $(D \rightarrow \bar{Q})$	12
t_{PHL} $(D \rightarrow \bar{Q})$	7
t_{PLH} $(C \rightarrow Q)$	15
t_{PHL} $(C \rightarrow Q)$	14
t_{PLH} $(C \rightarrow \bar{Q})$	16
t_{PHL} $(C \rightarrow \bar{Q})$	7
t_{su} (minimum setup time for D)	20
t_h (minimum hold time for D)	5
t_w (minimum pulse width on C)	20

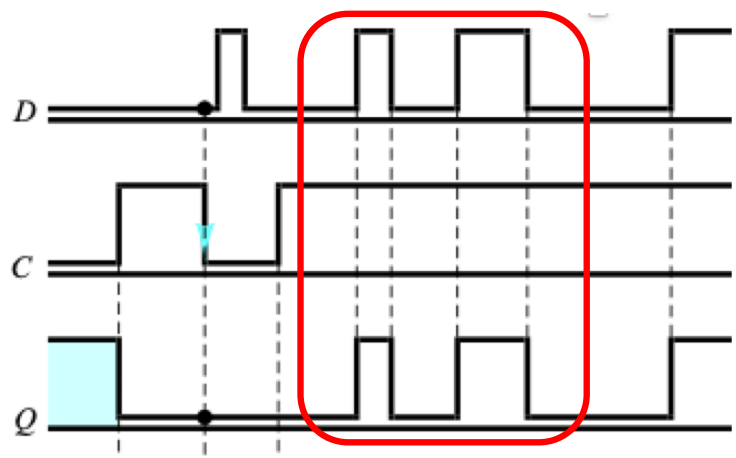
Flip-flops & Triggering

- CMSC313
- Dr. Kidd

Flip-Flops Motivation

Problem with Latches (1)

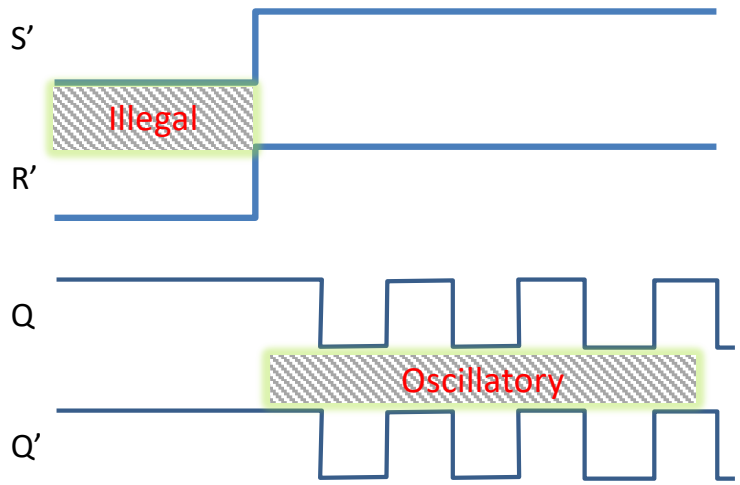
- Problem#1: Latches not appropriate for synchronous sequential circuits.
  - This is a D-latch
  - When enabling (C) is active, input directly gated into output
  - So any fluctuation in input is reflected in the output.
  - We want to control when this happens



Problem with Latches (1)

Problem#2: Using only latches <u>may</u> result in a oscillatory circuit changing state back and forth

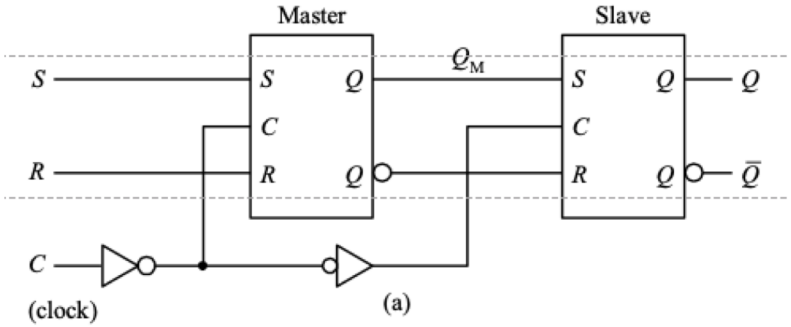
- Undefined state
- Continues even when (S,R) = (0,0)



The Solution: Flip-flops

Solution: Restrict when state changes are allowed by disconnecting the input from the output

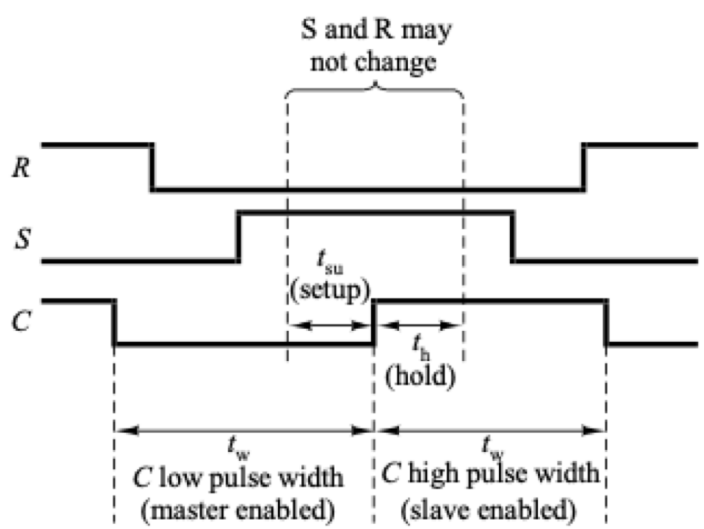
- Latches put in a Master-Slave configuration



- Master latch fluctuates like the normal latch
- Slave disconnects the output states (Q, Q') from the fluctuating inputs (S, R)
- Output states only change when Master is disconnected from inputs by the clock

Aside: What Constitutes a Flip-flop?

- Usage inconsistent
- Definitions vary
  - Some define a latch as a flip-flop
  - Some define flip-flops as being only edge triggered.
    - SR Master-slave are then excluded

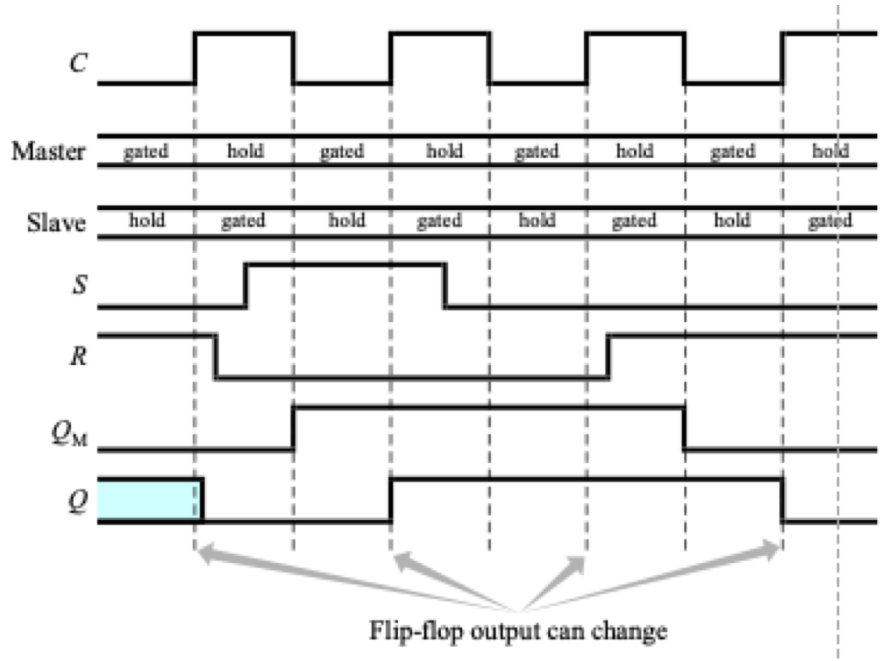


- Best definition is what is presented here
  - Output is isolated from the changes in the input
  - Includes both pulse and edge triggered
    - Pulse triggering sometimes referred to as 'level' triggering
  - Latches are not included

Triggering

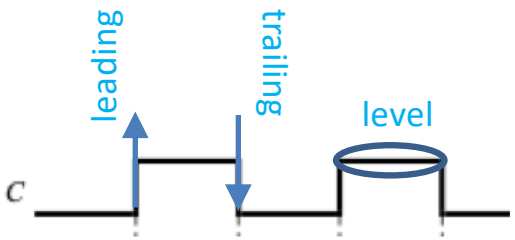
Triggering

- Triggering is simply when a module, e.g. a flip-flop, is 'active'
  - Meaning when the module can change its next state / value (e.g. output)
    - Next state is Q*
  - Usually used with flip-flops but can apply to other devices



Triggering

- Two types of triggering
  - Pulse / Level
    - High or Low
  - Edge
    - Leading or Trailing



- Want the triggering to occur over the shortest interval possible
  - Edge triggering obviously very short
  - Level triggering wants a short pulse
    - Minimizes the interval for which the output state can vary with the input state
    - This is why level triggering is also called pulse triggering

The J-K Flip-flop

J-K Flip-flop: Motivation

- S-R flip-flop has invalid state
  - S=1 & R=1
  - Result is indeterminate
- Better if there wasn't an invalid state
- J-K flip-flop eliminates invalid state.

J	K	Q	C	Q^*	
0	0	0	⎍	0	Hold
0	0	1	⎍	1	
0	1	0	⎍	0	Reset
0	1	1	⎍	0	
1	0	0	⎍	1	Set
1	0	1	⎍	1	
1	1	0	⎍	1	Toggle
1	1	1	⎍	0	

- S-R invalid state made into a toggle behavior
  - J=1 & K=1

Why not always use Flip-flops?

Latch	Flip-Flop
The latch is transparent – because input is directly connected to output when enable is high. It means Latch is sensitive to pulse duration (also called soft barrier)	Flip-flop is a pair of latches (master and slave flop). Flip-flop is sensitive to pulse transition. The signal only propagates through on the rising/falling edge (also called hard barrier)
Less Area (less gates)	More Area (more gates) because flip-flop contains two latches.
Less Power (less gates)	More Power (more gates)
Require more tool manipulation and more hand-calculations to verify that they meet timing	Easy to check design timing using Static Timing Analysis (STA) tools
Cycle-borrowing to gain more setup time on the next register stage, as long as each loop completes in one cycle	Data launches on one rising edge, so it must set up before the next rising edge. If it arrives late, the system fails. If it arrives early, time is wasted due to hard edges in Flops
To meet the timing in the design, Designers consider latches to adjust timing mismatch.	
For ASICs with large clock skew, latches have substantial benefits for reducing the clock period	Even for the high-speed pulsed flip-flops with zero setup time, as they are not transparent, the impact of the clock skew is not reduced
Level-sensitive latches reduce the impact of the inaccuracy of wire load models and process variation.	Flip-flops demands for the highly accurate wire load model and process
In DFT, Latches needed as a lockup state at the clock domain crossings in the scan chain to avoid unpredictable behavior	In DFT, use flops that can be scanned (controllable and observable)
In FPGA, level-sensitive transparent latches should be avoided in FPGAs	In FPGA, edge-sensitive flip-flops are used exclusively. Timings analysis is more appropriate with flops for FPGA tools
Circuit analysis is complex. You may see last minutes timing mismatch surprises at the implantation stage.	Circuit analysis is easy
High-speed microprocessor designs typically use master-slave latches instead of flip-flops so that logic can be added between the rising and falling clock edges.	The most commonly used flop in the design world is D type flip-flop.
Most of these companies have written their own specialized STA tools to verify latch-based designs.	FSM implementation mostly involves D Flip-flops due to a minimum number of logic gates and lesser cost as compared to other types of flip-flops.
For non-timing-critical configuration registers, latches work great, due to fewer gates and less power consumption	For non-power aware design, Flip flops are preferred over Latches
The latch is an asynchronous block. Therefore you must ensure that the combinational functions, which generate input signals for the latch, are race-free. Otherwise, they may generate glitches, which may be latched, causing hazards in your system.	A flip-flop, on the other hand, is edge-triggered and only changes state when a control signal goes from high to low or low to high
Latch based design is noisy because any noise in the enable signal disrupts the latch output easily.	

Available Flip-flops

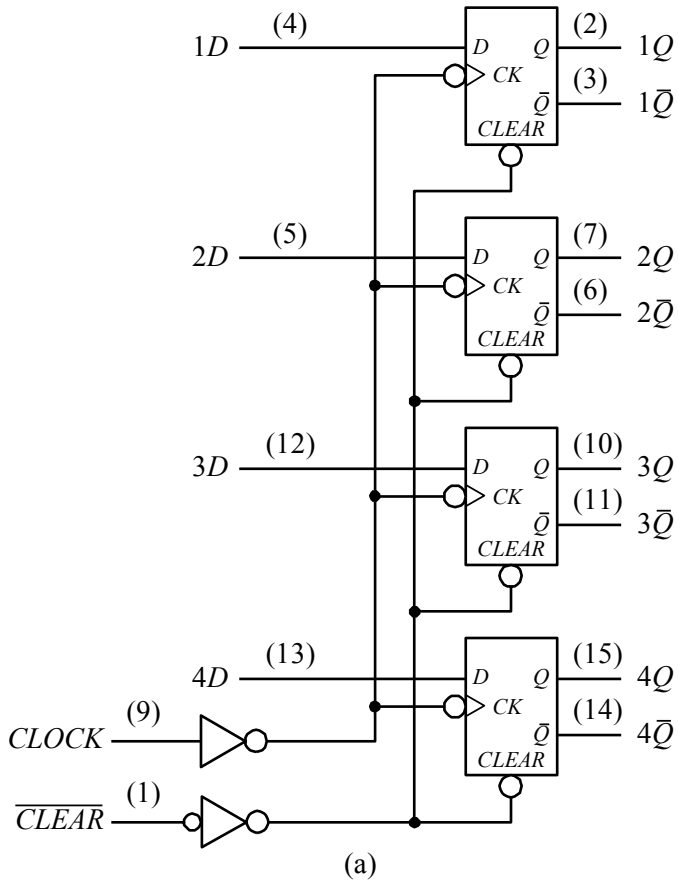
J-K Flip-flop: Motivation

- S-R flip-flop has invalid state
  - S=1 & R=1
  - Result is indeterminate
- Better if there wasn't an invalid state
- J-K flip-flop eliminates invalid state.

J	K	Q	C	Q^*	
0	0	0	⎍	0	Hold
0	0	1	⎍	1	
0	1	0	⎍	0	Reset
0	1	1	⎍	0	
1	0	0	⎍	1	Set
1	0	1	⎍	1	
1	1	0	⎍	1	Toggle
1	1	1	⎍	0	

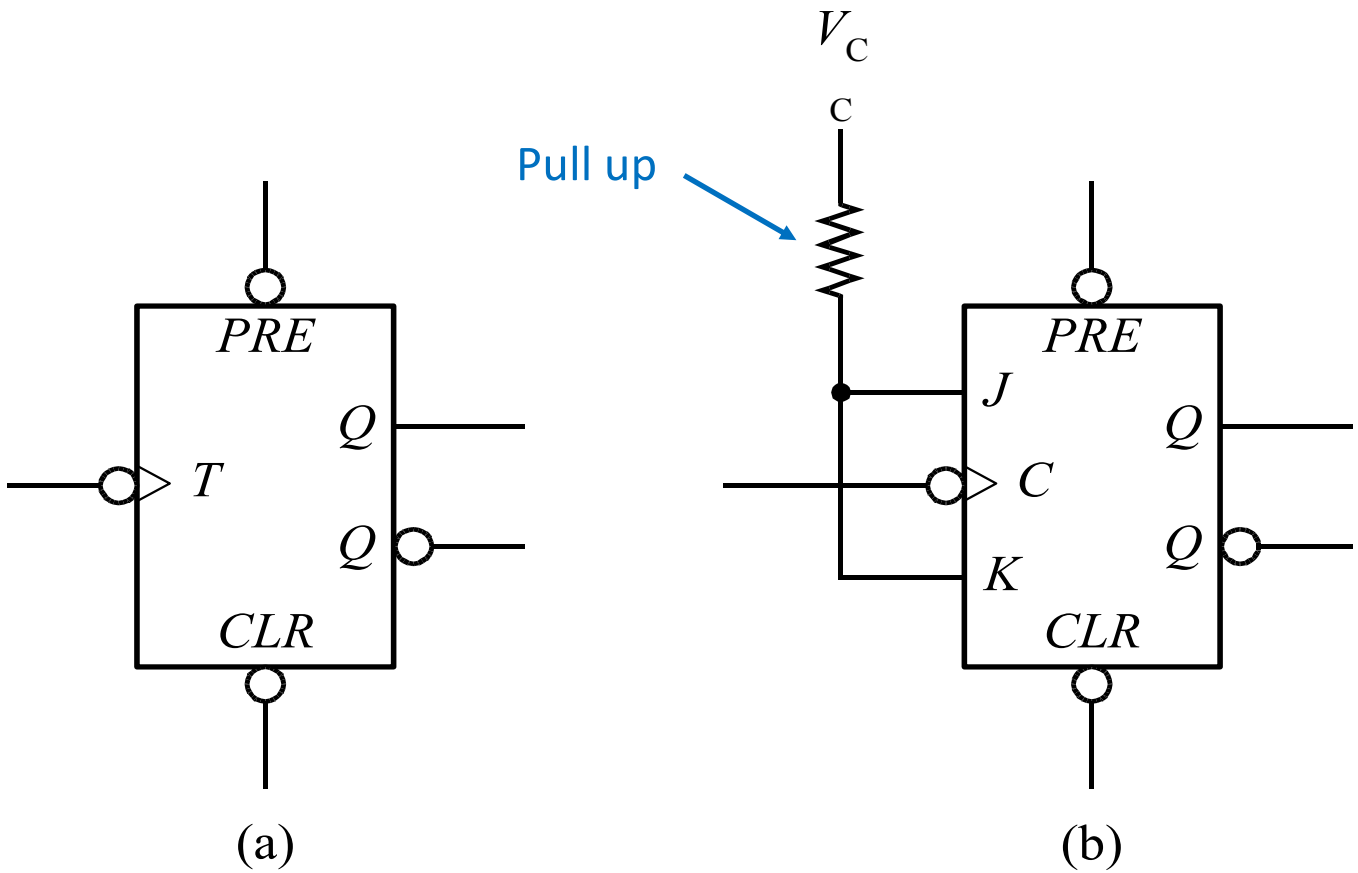
- S-R invalid state made into a toggle behavior
  - J=1 & K=1

SN74175 Positive-Edge-Triggered D Flip-Flop



(a)

Negative-Edge-Triggered T Flip-Flop



Summary

Summary of Latch and Flip-Flop Characteristics

- Latches
  - Less complicated
  - Less precise
    - Slower circuits
  - Smaller footprint
  - Usage
    - Slower circuits
    - Infrequent state changes
- Flip-flops
  - More complicated
  - More precise
    - faster circuits
  - Larger footprint
  - Usage
    - Fast digital circuits
    - Frequent state changes

Table 6.3 Summary of Latch and Flip-flop Characteristics

Device	Characteristic Equation
SR Latch	$Q^* = S + R'Q$
Gated SR Latch	$Q^* = SC + Q'R + C'Q$
D Latch	$Q^* = DC + C'Q$
SR Flip-flop	$Q^* = S + R'Q$
D Flip-flop	$Q^* = D$
JK Flip-flop	$Q^* = K'Q + JQ'$
T Flip-flop (edge-triggered)	$Q^* = Q'$
T Flip-flop (clocked)	$Q^* = TQ' + T'Q$

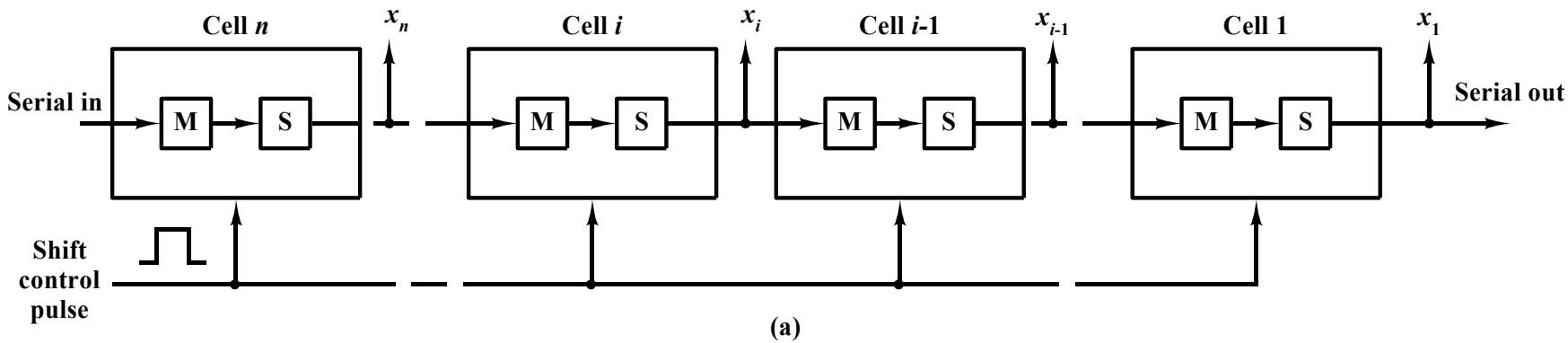
Sequential Digital Circuits
Shift Registers and Counters
Cumulative Adders

- CMSC313
- Dr. Kidd

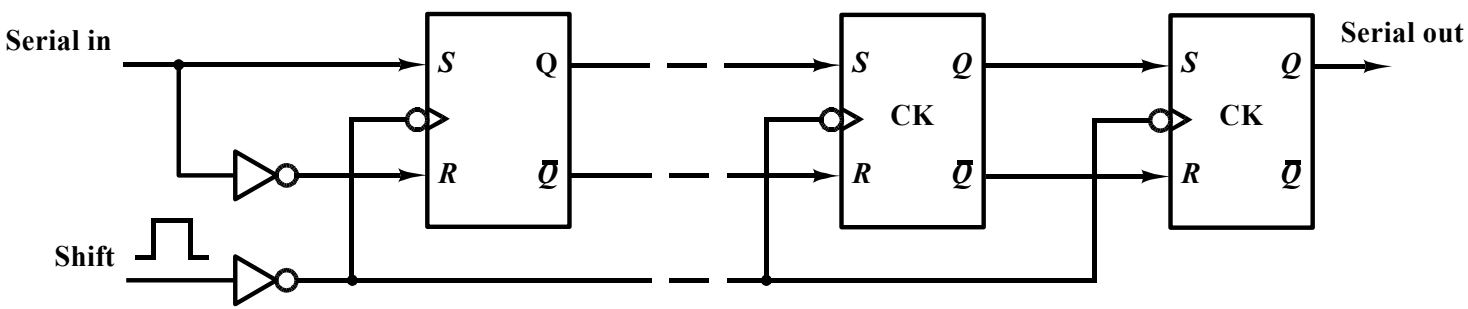
Acknowledgements

- Parts of these lectures are taken or derived from one or more of the following:
- Harris and Harris, Digital Design and Computer Architecture, RISC5 Edition
- Victor P. Nelson, Carroll, Nagle, Irwin, Digital Logic Circuit Analysis and Design.
- M. Morris Mano, Digital Logic and Computer Design.

Serial-in, Serial-out Shift Register

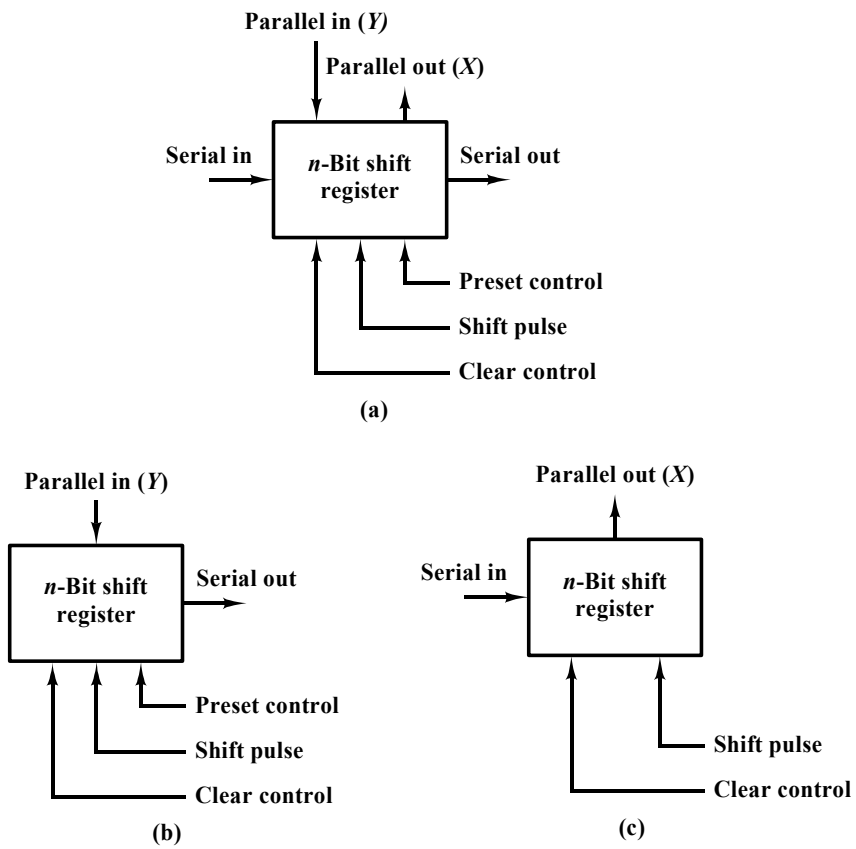


(a)



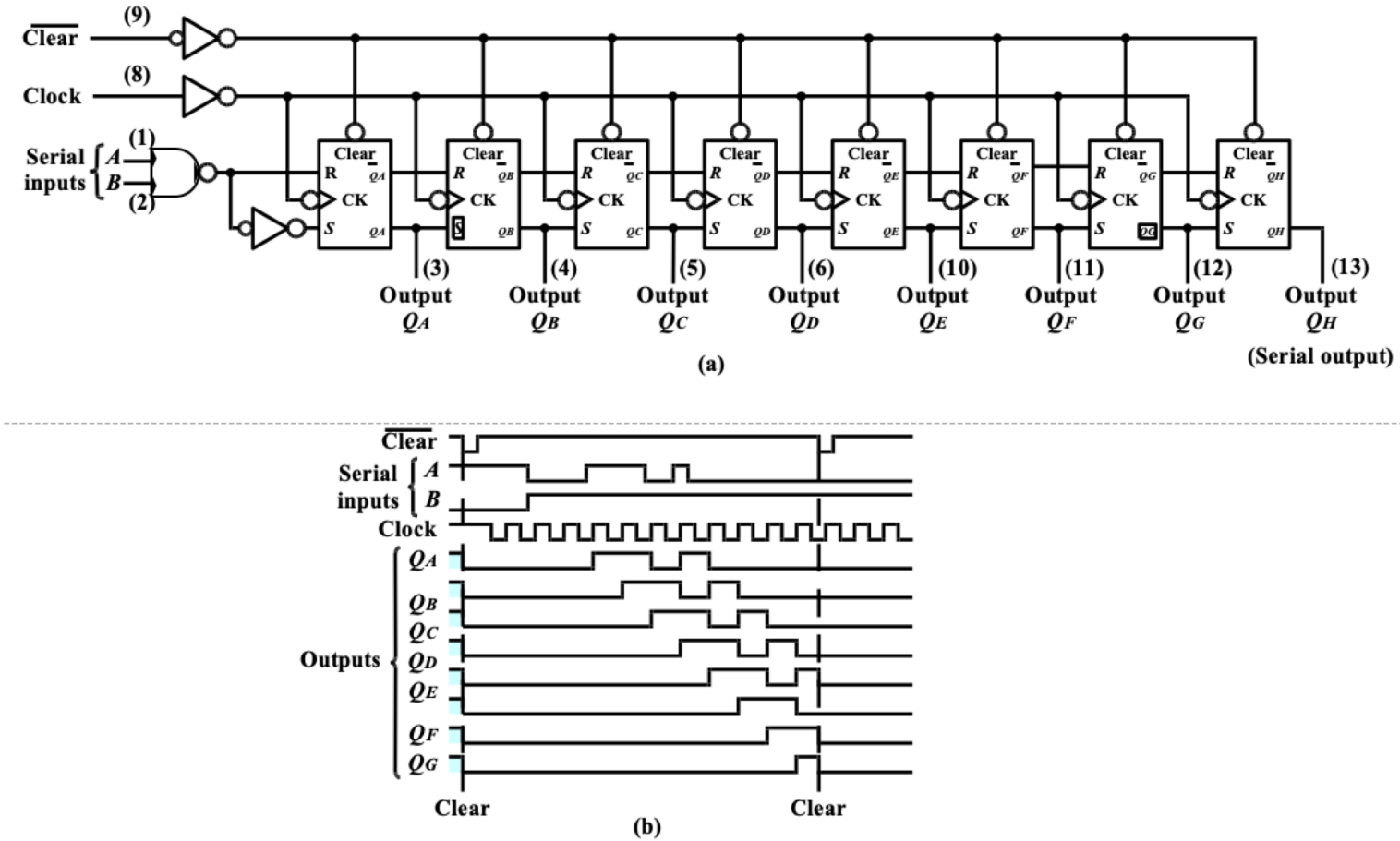
(b)

Generic Register

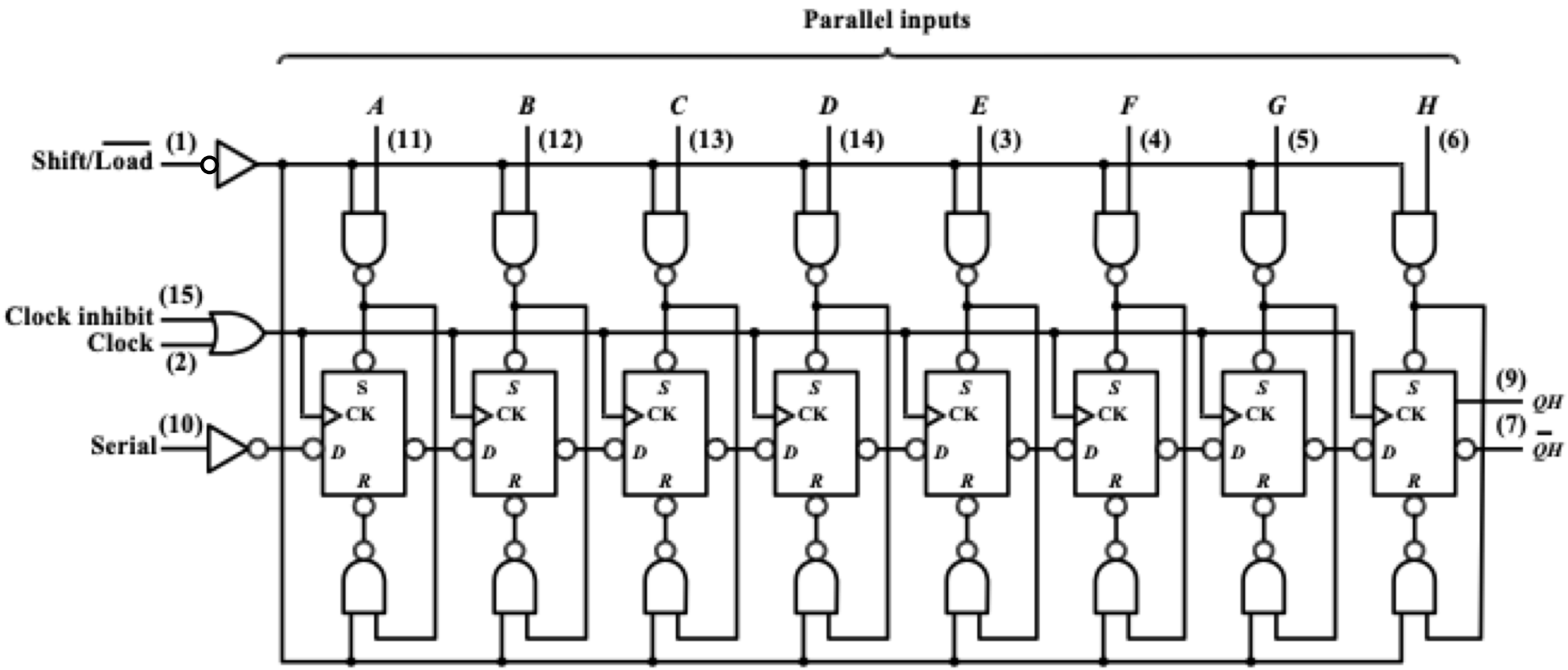


- Why not always use parallel register?
  - It has to do with what circuitry is connected to the register
  - Parallel circuit is more complicated generally
- Input circuitry might be less complicated and serial
  - Use serial in
- Output circuitry might be less complicated and serial
  - User serial out
- Output / Input might be inherently serial
  - E.g. Serial transfer such as USB

SN74164 Serial-in, Serial-out Shift Register



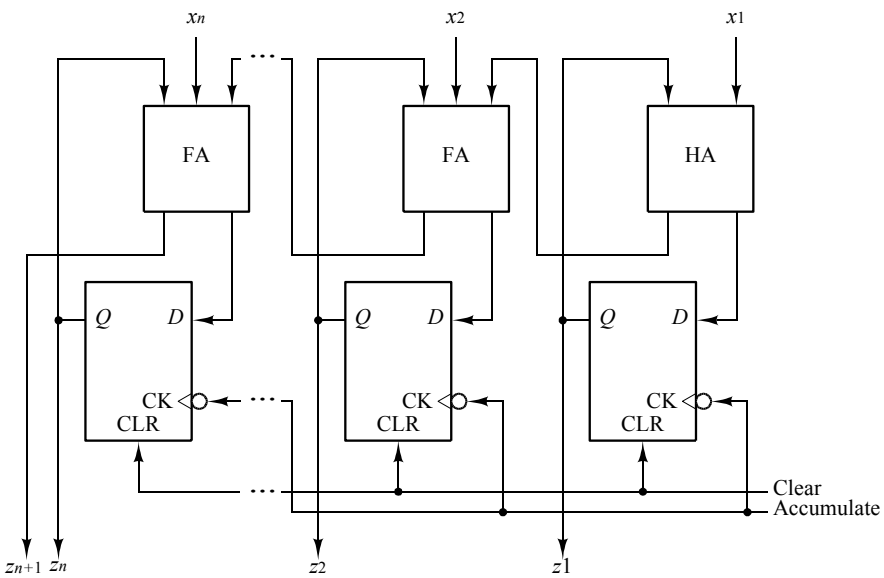
SN74165 8-bit Serial-In, Parallel-In, Serial-out Shift register



(a)

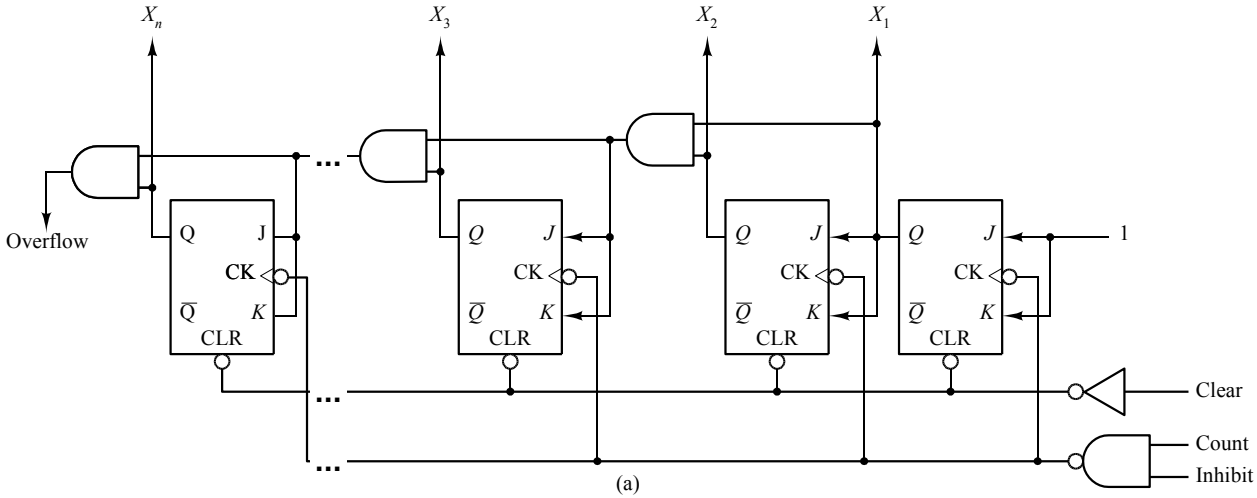
Parallel Accumulator

- Add still has to propagate from LSB to MSG
  - Asynchronously
  - Signal captures result in accumulator register
- Accumulates
  - Adds series of successive numbers
    - x1, x2, x3, …
- ALU specs will tell
  - Carry latency
  - Result latency
- How much time is needed before we can capture accumulator?



(b)

Synchronous Binary Counter



(a)

X_n	X_3	X_2	X_1	
0	0	0	0	
0	0	0	1	
0	0	1	0	
0	0	1	1	
0	1	0	0	
1	1	1	0	
1	1	1	1	← Recycles
0	0	0	0	
0	0	0	1	
0	0	1	0	

(b)

Asynchronous Down Counter

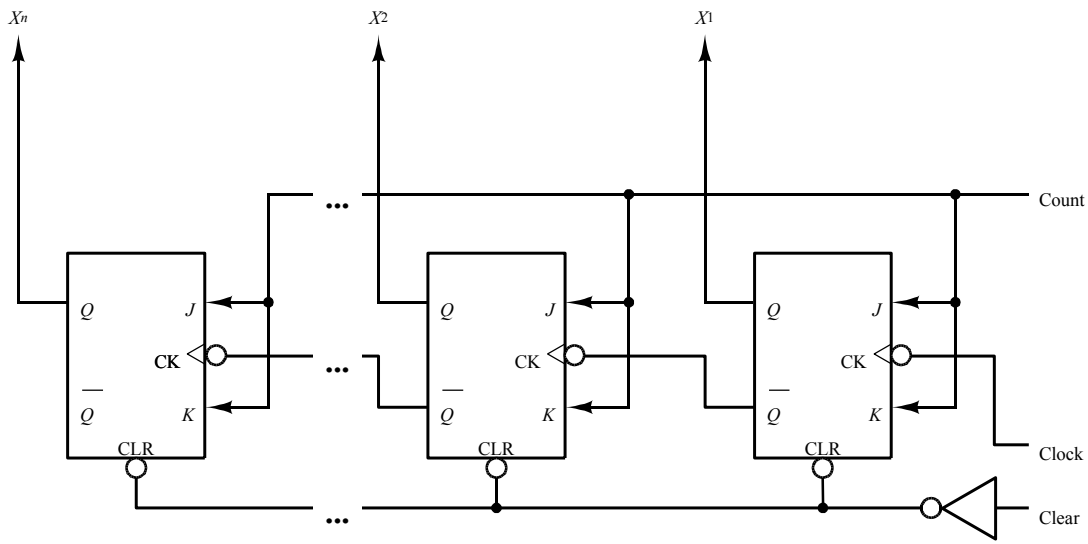
X_n	...	X_3	X_2	X_1
1	...	1	1	1
0	...	0	0	0
0	...	0	0	1
0	...	0	1	0
0	...	0	1	1
0	...	1	0	0

Up count mode

X_n	...	X_3	X_2	X_1
0	...	0	0	0
1	...	1	1	1
1	...	1	1	0
1	...	1	0	1
1	...	1	0	0
1	...	0	1	1

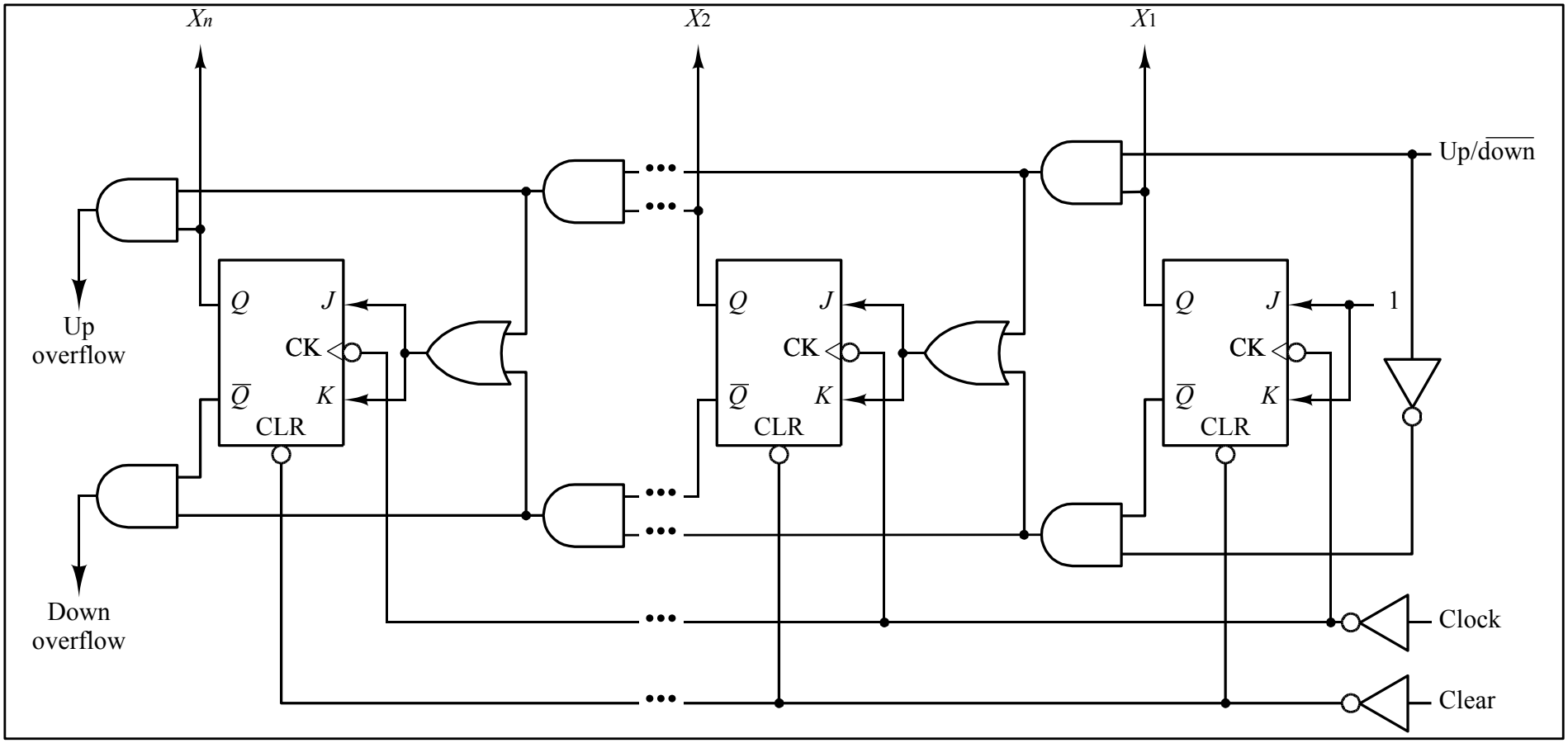
Down count mode

(a)

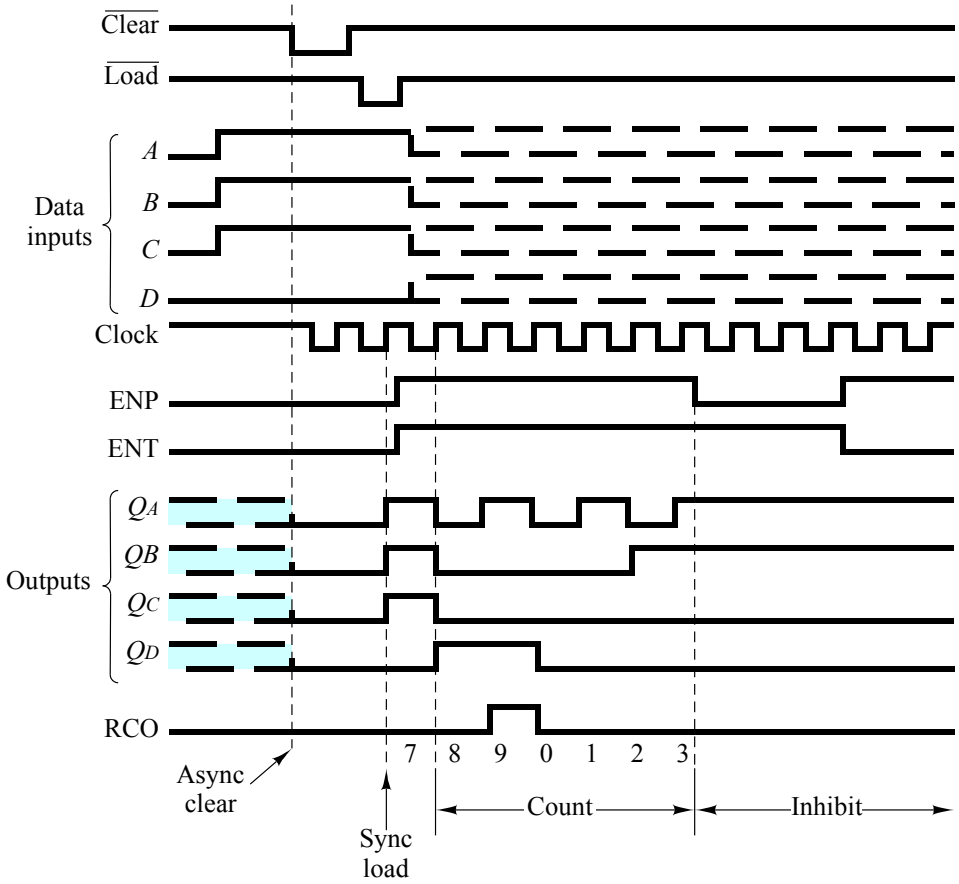


(b)

Synchronous Up/Down Counter

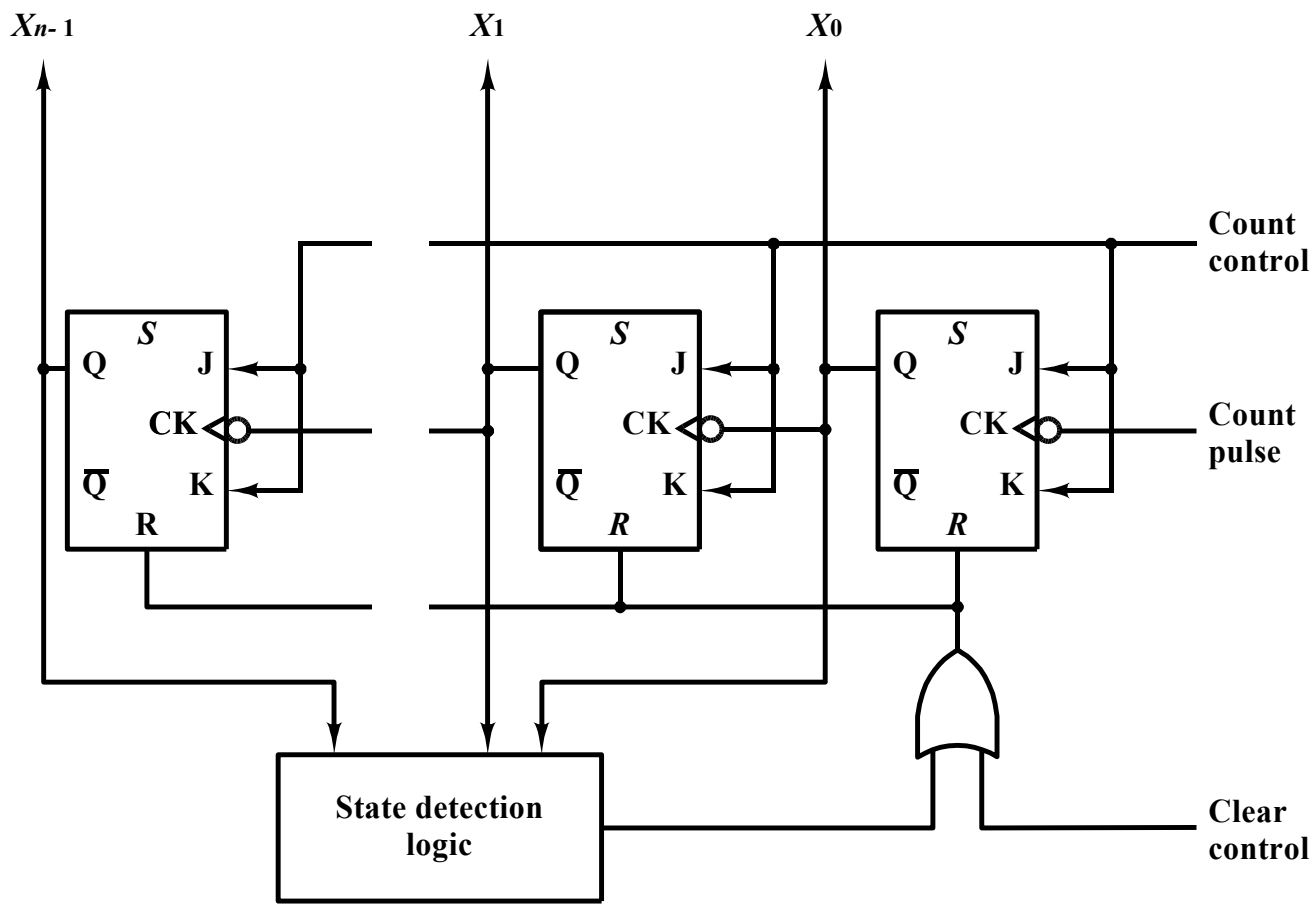


SN74160 Timing Diagram



(d)

Modulo-N Asynchronous Counter



When Synchronous vs Asynchronous?

- Asynchronous
  - Very fast
  - Need final output and don't care about intermediate states
    - Similar to ripple adder
  - Smaller footprint
  - Can have timing issues with surrounding circuit
- Synchronous
  - Precise timing
  - Need intermediate states
  - Coordination with surrounding circuitry and modules
    - Computer processor
  - Larger footprint
  - Fewer timing issues